Technical Reference

Stack, data model, modules, REST API, and configuration

1. Stack

Layer	Technology
API	FastAPI with Uvicorn; Pydantic v2 schemas
Orchestration	LangGraph compiled StateGraph with a SQLite checkpointer
Reasoning	Google Gemini via google-genai (function calling and synthesis)
Vector store	Qdrant client, 768-dimensional cosine collections (in-memory by default)
Relational store	SQLAlchemy 2.0 asyncio; SQLite locally, PostgreSQL supported
Frontend	Streamlit, communicating only through authenticated REST calls
Auth	PBKDF2-HMAC-SHA256 password hashing; HMAC-SHA256 signed JWTs
Tests	pytest with pytest-asyncio

LIMITATION
sqlalchemy[asyncio] requires greenlet, and local PDF upload requires pypdf. Both are declared in requirements.txt; installing SQLAlchemy without the asyncio extra will fail at import time.

2. Relational schema

Defined in `app/db/models.py`.

Table	Purpose	Notable columns
users	Accounts and roles	user_id, email, password_hash, role, is_active
tool_definitions	The authoritative tool contract	name, namespace, method, endpoint_path, path_params, query_params, request_body_schema, risk_level, approval_policy, permitted_roles, dependencies
agent_runs	One row per turn	run_id, user_id, session_id, query, status, approval_status, tools_used, execution_trace, final_response, duration_ms
execution_log	One row per tool attempt	user_id, request_id, tool_name, status, input_args, response_payload, latency_ms, error_message
token_usage	Model tokens per request	request_id, user_id, model, input_tokens, output_tokens, total_tokens
approval_requests	Human decisions on gated actions	approval_id, run_id, user_id, tool_name, status, reason, decided_by
knowledge_sources	Indexed documents	source_id, name, source_type, domain, chunk_count, status
mcp_servers	Registered MCP servers	server_id, name, server_type, command, args, discovered_tools_count
conversations	Conversation records	conversation_id, user_id, title, messages

JOIN KEY
agent_runs.run_id, token_usage.request_id and execution_log.request_id all carry the same per-turn identifier. Joining on it is what lets a task be shown with its own token cost and its own tool-call rows.

3. Module map

Path	Responsibility
app/main.py	FastAPI application, endpoints, RBAC dependencies, startup seeding
app/config.py	Pydantic settings loaded from environment and .env
app/core/security.py	Password hashing, JWT issue and verify, role dependencies
app/graph/workflow.py	Graph definition, checkpointer, run and resume entry points
app/graph/nodes.py	The ten pipeline nodes and the deterministic fallback planner
app/graph/state.py	The typed AgentState carried between nodes
app/services/llm_service.py	Gemini function calling, grounded synthesis, token extraction
app/services/tool_registry.py	Dual-index catalog: relational contract plus vector index
app/services/tool_retriever.py	Two-stage hybrid retrieval with RRF and dependency expansion
app/services/execution_engine.py	Validation, idempotency, dispatch (mock / HTTP / MCP), audit
app/services/ingestion.py	Postman collection parsing into tool contracts
app/services/document_service.py	Document and URL ingestion, chunking, vector indexing
app/services/mcp_client.py	MCP server discovery and stdio JSON-RPC bridge
app/meta_tools/	The pinned RAG and System Search tools, plus general utilities
frontend/	Streamlit screens and the REST client

4. REST API

All routes except `/health` and `/auth/login` require a bearer token. Routes under `/admin/` require the administrator role.

Method and path	Purpose
GET /health	Liveness probe
POST /auth/login	Exchange credentials for a JWT
GET /auth/me	Current account and role
POST /chat	Run one agent turn
GET /runs/{id}	Inspect a run (own runs, or any for an administrator)
POST /runs/{id}/approval	Approve or reject a suspended run; resumes the graph
GET /tools	Browse the catalog
POST /admin/tools	Register a tool
PUT /admin/tools/{id}	Update a tool
PATCH /admin/tools/{id}/status	Enable or disable a tool
POST /admin/tools/{id}/validate	Validate a tool's schema
POST /admin/tools/{id}/test	Mock-execute a tool
GET /sources	Browse indexed documents
POST /admin/sources/upload	Ingest text or an extracted document

Method and path	Purpose
POST /admin/sources/url	Ingest a documentation URL
DELETE /admin/sources/{id}	Remove a source
GET /usage/me	Own aggregate usage
GET /usage/me/activity	Own tasks, each with its tokens and tool-call rows
GET /logs	Execution log rows (own rows; all rows for an administrator)
GET /admin/users	User directory with aggregates
GET /admin/users/{id}/activity	A user's tasks with tokens and tool-call rows
GET /admin/users/{id}/executions	A user's run history
GET /admin/users/{id}/tokens	A user's token records
GET /admin/usage/summary	Platform totals
GET /mcp/servers	Registered MCP servers
POST /mcp/servers/discover	Discover tools a server exposes
POST /mcp/servers/register-tools	Index discovered MCP tools into the catalog
POST /mcp/tools/execute	Invoke an MCP tool directly

5. Configuration

Copy `.env.example` to `.env`. Defaults run the platform with no external services.

Variable	Default	Notes
GEMINI_API_KEY	(empty)	Without it the deterministic planner is used and no tokens are recorded
GEMINI_MODEL	gemini-2.5-flash	Model used for planning and synthesis
EMBEDDING_MODEL	text-embedding-004	Falls back to a local deterministic embedding
DATABASE_URL	sqlite+aiosqlite:///./agentic_system.db	PostgreSQL via asyncpg is supported
QDRANT_USE_MEMORY	true	Set false to use a Qdrant server at QDRANT_HOST / QDRANT_PORT
EXECUTION_MODE	mock	mock, sandbox or live; live also needs PayPal credentials
PAYPAL_CLIENT_ID / SECRET	(empty)	Absent, dispatch stays in mock mode
MAX_AGENT_ITERATIONS	10	Upper bound on tool calls per turn
RETRIEVAL_TOP_K	6	Base shortlist size before pinning and dependency expansion
JWT_SECRET_KEY	dev default	Must be replaced outside local development
SMTP_*	(empty)	Absent, email dispatch is simulated

6. Tests

Run `python -m pytest tests/ -q`. The suite runs offline: no API key, no network, no external services.

Suite	Covers
tests/unit/test_llm_planner.py	Schema translation to function declarations, shortlist enforcement, token extraction, fallback on model failure, graph wiring (Gemini stubbed)
tests/integration/test_approval_resume.py	Suspend and resume, rejection, single-use decisions, survival across a restart, conversation memory
tests/integration/test_paypal_flows.py	End-to-end turns for the task's scenarios
tests/unit/	Auth, permissions, retrieval, ingestion, meta-tools

7. Observability

Every turn emits a stage-by-stage timeline, rendered in the chat screen, alongside the audit tables. LangSmith tracing is wired in and optional: with `LANGSMITH_API_KEY` set, LangGraph contributes a span per pipeline node and `app/core/tracing.py` adds explicit spans around the two reasoning calls, carrying the prompt, the candidate shortlist, the selected tool and the token counts. With no key, tracing is off and the decorator is a no-op, so nothing in the call path depends on it.

8. Known limitations

- **The live model path is not covered by automated tests.** Planner and synthesis are tested against a stubbed SDK; running with a real key is a manual step.
- **Embeddings fall back to a local hash with no API key.** Document retrieval then relies on token overlap and will not match synonyms. With a key configured, both the tool index and the knowledge base use Gemini

embeddings.

- **Changing embedding provider requires a reindex.** Content indexed with one provider will not match queries embedded by the other; affected sources must be deleted and re-ingested.
- **Qdrant runs in memory by default**, so the tool index is rebuilt on each start.
- **13 of 73 catalog entries carry no parameters**, being endpoints the seeded collection defines without path, query or body fields.
- **The deterministic fallback router recognises only its encoded intents**, so a document question with no trigger word can miss the RAG tool. The model-driven planner routes on meaning instead.
- **Schema changes use ad-hoc SQLite ALTER statements** rather than a migration tool.